code for processing peptides

Aaron McKenna

June 13, 2014

Read in the data from the three replicate files. These are exported from excel as txt (tab seperated) after data cleanup and curation that Nate Camp setup:

```
bio1 <- read.delim("~/Desktop/UW/yadlin/new_data/repA.txt")
bio2 <- read.delim("~/Desktop/UW/yadlin/new_data/repB.txt")
bio3 <- read.delim("~/Desktop/UW/yadlin/new_data/repC.txt")
```

Make some representative (proportional) Venn diagrams, mostly just to test it out (and to understand if one set is limiting our overlap):

```
require(venneuler)

## Loading required package:  venneuler
## Warning:  package 'venneuler' was built under R version 3.0.2
## Loading required package:  rJava
## Warning:  package 'rJava' was built under R version 3.0.2

v <- venneuler(c(A = length(unique(bio1$peptide)), B = length(unique(bio2$peptide)), C = length(unique(b
    `A&B` = length(intersect(bio1$peptide, bio2$peptide)), `A&C` = length(intersect(bio1$peptide,
        bio3$peptide)), `B&C` = length(intersect(bio2$peptide, bio3$peptide)), `A&B&C` = length(intersec
        bio2$peptide), bio3$peptide))))
v$labels <- c("replicate 1", "replicate 2", "replicate 3")
plot(v)

print(paste("A&C", length(intersect(bio1$peptide, bio3$peptide))))

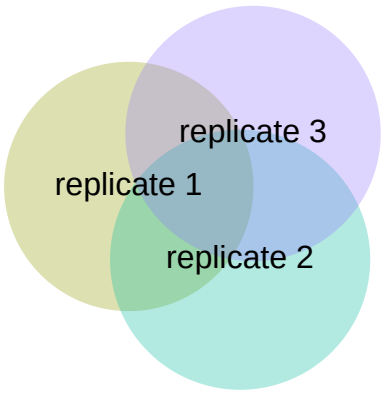
## [1] "A&C 1337"

print(paste("B&C", length(intersect(bio2$peptide, bio3$peptide))))

## [1] "B&C 1727"

print(paste("A&B&C", length(intersect(intersect(bio1$peptide, bio2$peptide), bio3$peptide))))

## [1] "A&B&C 1068"
```



Now subset the data to the overlapping peptides common to all the replicates, median center AFTER subsetting. This seems to be the right thing to do, considering that the data is already normalized within replicates, and this median for the overlap peptides could be very skewed by one replicate having many low quality false positives.

```
# intersect the data
intersection_peptides <- intersect(intersect(bio1$peptide, bio2$peptide), bio3$peptide)
bio1 <- bio1[is.element(bio1$peptide, intersection_peptides), ]
bio2 <- bio2[is.element(bio2$peptide, intersection_peptides), ]
bio3 <- bio3[is.element(bio3$peptide, intersection_peptides), ]

# median center --after-- the subset
bio1[, 4:11] <- sweep(bio1[, 4:11], 2, apply(bio1[, 4:11], 2, median), `/`)
bio2[, 4:11] <- sweep(bio2[, 4:11], 2, apply(bio2[, 4:11], 2, median), `/`)
bio3[, 4:11] <- sweep(bio3[, 4:11], 2, apply(bio3[, 4:11], 2, median), `/`)
```

Setup the tukey test, testing if the change seen in a peptide across each pairing of its replicates is significant. This is a much better approach than our previous homebrew ANOVA tests (thanks Tony!)

```
# tukey test - commands borrowed from
# http://www.flaviobarros.net/2013/06/13/anova-and-tukey-test-on-r/
library(reshape)

## Loading required package:  plyr
## Warning:  package 'plyr' was built under R version 3.0.2
##
## Attaching package:  'reshape'
##
## The following object is masked from 'package:plyr':
##
##     rename, round_any
```

```
tukey.single.row <- function(my.data) {
    sub.frame <- cbind(as.numeric(my.data[4:11]), as.numeric(my.data[15:22]), as.numeric(my.data[26:33])
    # print(sub.frame)
    colnames(sub.frame) <- c("rep1", "rep2", "rep3")
    rownames(sub.frame) <- c("tp1", "tp2", "tp3", "tp4", "tp5", "tp6", "tp7", "tp8")
    ddd <- melt(sub.frame)
    colnames(ddd) <- c("timepoint", "replicate", "value")
    a1 <- aov(formula = ddd$value ~ ddd$timepoint + ddd$replicate)
    posthoc <- TukeyHSD(x = a1, "ddd$timepoint", conf.level = 0.95)
    return(posthoc$`ddd$timepoint`)
}
```

Run the Tukey test on each peptide, comparing each timepoint to every other. Attach this data to the master table:

```
# setup a table containing all the data, merged at the peptide level
total.bio <- merge(bio2, bio3, by.x = "peptide", by.y = "peptide", suffixes = c(".rep2", ".rep3"))
total.bio2 <- merge(bio1, total.bio, by.x = "peptide", by.y = "peptide")

# find the minimum value, median, and record all comparisons in our master table
total.bio2$tp.pv.min <- as.numeric(apply(total.bio2, 1, function(x) {
    min(tukey.single.row(x)[, "p adj"])
}))
total.bio2$tp.pv.med <- as.numeric(apply(total.bio2, 1, function(x) {
    median(tukey.single.row(x)[, "p adj"])
}))
total.bio2 <- cbind(total.bio2, t(apply(total.bio2, 1, function(x) {
    tukey.single.row(x)[, "p adj"]
})))
```

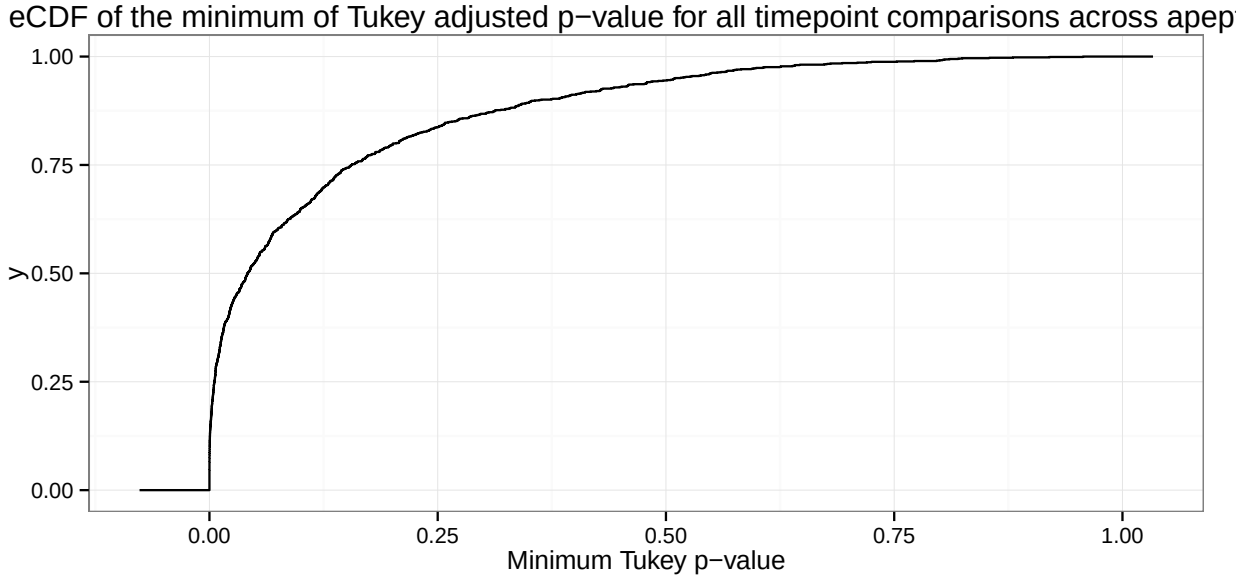
Plot the empirical cumulative distribution of the median, mean, and all Tukey p-values values:

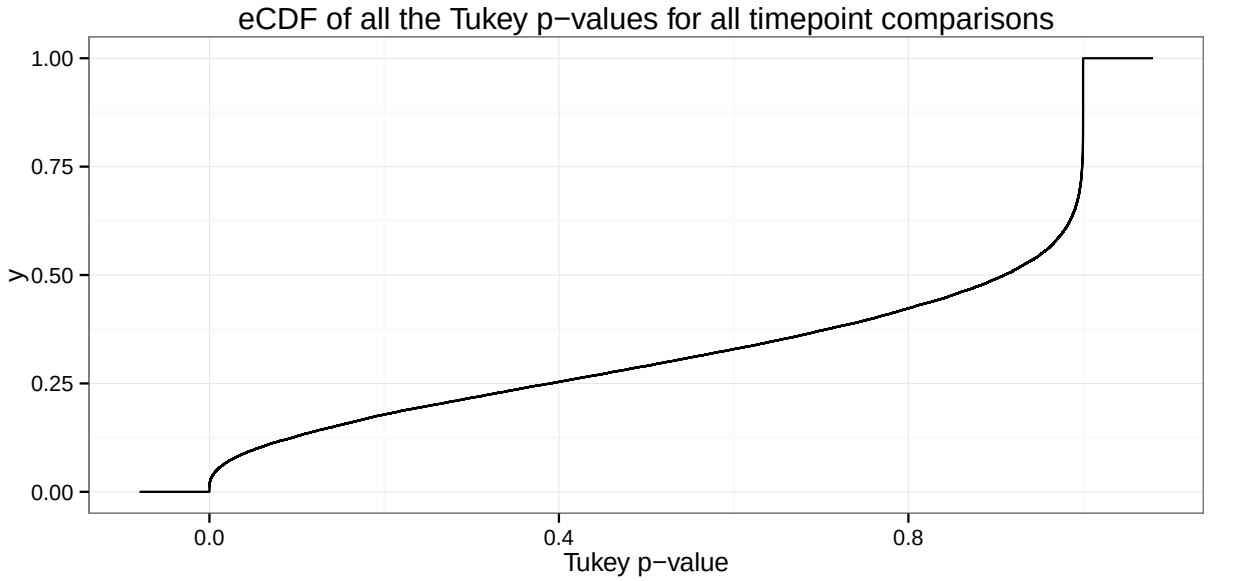
```
library(ggplot2)
qplot(total.bio2$tp.pv.min, stat = "ecdf", geom = "step") + ggtitle("eCDF of the minimum of Tukey adjust
    theme_bw() + xlab("Minimum Tukey p-value")
qplot(total.bio2$tp.pv.med, stat = "ecdf", geom = "step") + ggtitle("eCDF of the median of Tukey p-value
    theme_bw() + +xlab("Median Tukey p-value")

## Error:  invalid argument to unary operator

qplot(melt(total.bio2[, 37:64])$value, stat = "ecdf", geom = "step") + ggtitle("eCDF of all the Tukey p-
    theme_bw() + xlab("Tukey p-value")

## Using  as id variables
## Using  as id variables
```





Subset the larger table by the columns we'd like to output. This was more important when we didn't output most columns.

```
for.output <- total.bio2[, c(1, 2, 3, 35, 36, 4:11, 15:22, 26:33, 37:64)]
colnames(for.output) <- c("peptide", "protein", "gene.name", "minimum.tukey.p.value", "median.tukey.p.va
    "rep1.tp0", "rep1.tp2", "rep1.tp4", "rep1.tp8", "rep1.tp16", "rep1.tp32", "rep1.tp64",
    "rep1.tp128", "rep2.tp0", "rep2.tp2", "rep2.tp4", "rep2.tp8", "rep2.tp16", "rep2.tp32",
    "rep2.tp64", "rep2.tp128", "rep3.tp0", "rep3.tp2", "rep3.tp4", "rep3.tp8", "rep3.tp16",
    "rep3.tp32", "rep3.tp64", "rep3.tp128", "tp2-tp1", "tp3-tp1", "tp4-tp1", "tp5-tp1", "tp6-tp1",
    "tp7-tp1", "tp8-tp1", "tp3-tp2", "tp4-tp2", "tp5-tp2", "tp6-tp2", "tp7-tp2", "tp8-tp2",
    "tp4-tp3", "tp5-tp3", "tp6-tp3", "tp7-tp3", "tp8-tp3", "tp5-tp4", "tp6-tp4", "tp7-tp4",
    "tp8-tp4", "tp6-tp5", "tp7-tp5", "tp8-tp5", "tp7-tp6", "tp8-tp6", "tp8-tp7")

# force non-scientific output.  Some programs don't always parse mixed (scientific and
```

```
# not), and we're not dealing with crazy precision
options(scipen = 10000)

# find the medians of each timepoint at peptide
for.output$median.tp0 <- apply(for.output, 1, function(x) {
    median(as.numeric(c(x["rep1.tp0"], x["rep2.tp0"], x["rep3.tp0"])))
})
for.output$median.tp2 <- apply(for.output, 1, function(x) {
    median(as.numeric(c(x["rep1.tp2"], x["rep2.tp2"], x["rep3.tp2"])))
})
for.output$median.tp4 <- apply(for.output, 1, function(x) {
    median(as.numeric(c(x["rep1.tp4"], x["rep2.tp4"], x["rep3.tp4"])))
})
for.output$median.tp8 <- apply(for.output, 1, function(x) {
    median(as.numeric(c(x["rep1.tp8"], x["rep2.tp8"], x["rep3.tp8"])))
})
for.output$median.tp16 <- apply(for.output, 1, function(x) {
    median(as.numeric(c(x["rep1.tp16"], x["rep2.tp16"], x["rep3.tp16"])))
})
for.output$median.tp32 <- apply(for.output, 1, function(x) {
    median(as.numeric(c(x["rep1.tp32"], x["rep2.tp32"], x["rep3.tp32"])))
})
for.output$median.tp64 <- apply(for.output, 1, function(x) {
    median(as.numeric(c(x["rep1.tp64"], x["rep2.tp64"], x["rep3.tp64"])))
})
for.output$median.tp128 <- apply(for.output, 1, function(x) {
    median(as.numeric(c(x["rep1.tp128"], x["rep2.tp128"], x["rep3.tp128"])))
})
write.table(for.output, file = "replicate_output.txt", sep = "\t", quote = F, row.names = F)
```